Computer Networks

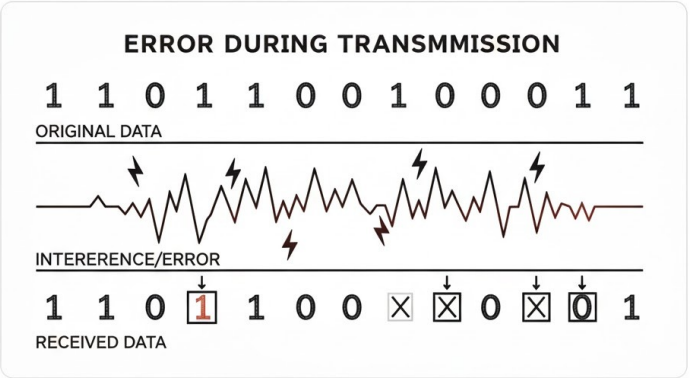
PPT-3 Error Detection and Correction

Anand
Department of Computer Science
Keshav Mahavidyalaya

Error Detection & Correction

Error detection in computer networks is a process of identifying errors that may occur during the transmission of data between devices in a network. These errors can be caused by various factors, such as noise, interference, **signal attenuation***, or hardware failures. The primary goal of error detection mechanisms is to ensure the integrity of the transmitted data and, if possible, to correct any errors that occur.

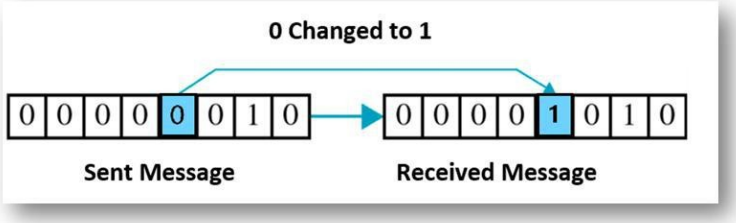
**Signal attenuation is the loss of signal strength as it travels through a medium or system*

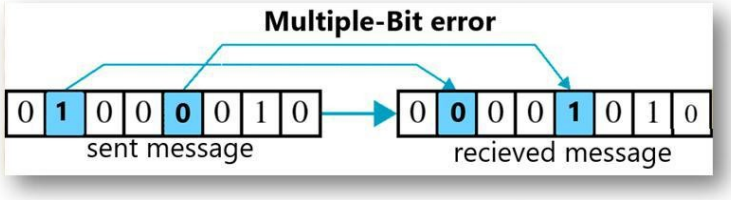


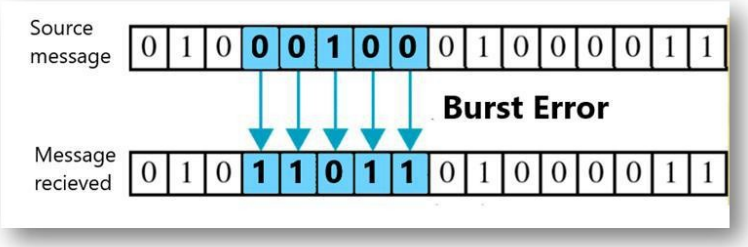
Types of Network Errors

- Single Bit Error
- Multiple Bit Error
- Burst Error

- Cable or Connection Faults
- Faulty Network Interface Cards (NICs)
- Router or Switch Failures
- Damaged or Faulty Wireless Access Points
- Power Supply Issues
- Signal Interference (Wireless Networks)
- Overheating of Network Devices
- Broken Ports or Connectors
- Hardware Incompatibility
- Firmware or Hardware Configuration Issues





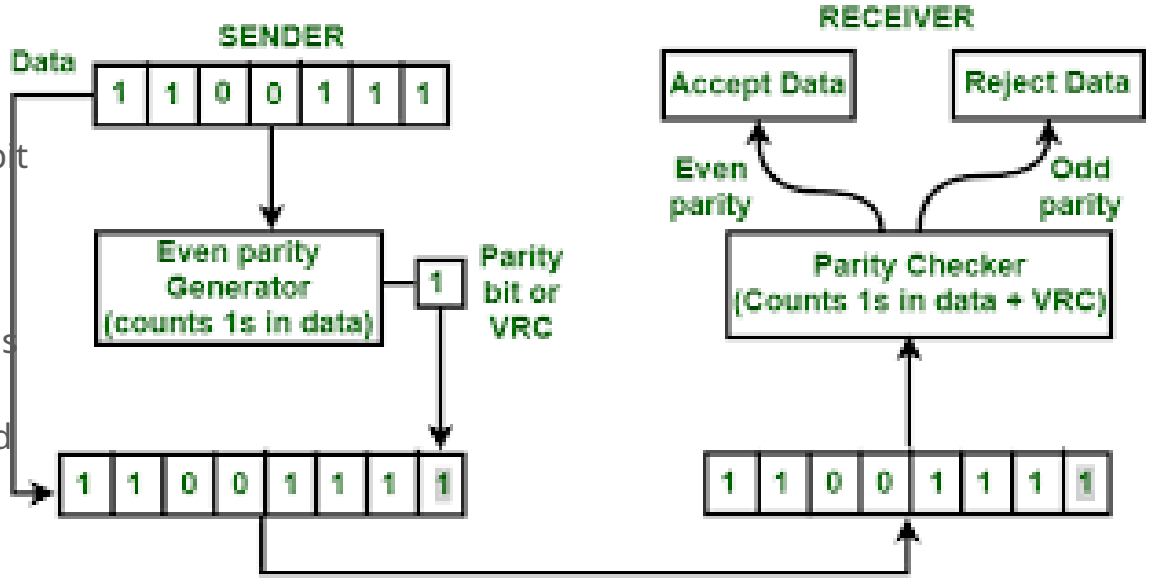


Methods for Error Detection

1. Parity / VRC
2. 2d Parity /LRC
3. Checksum
4. Cyclic Redundancy Check

Parity /VRC

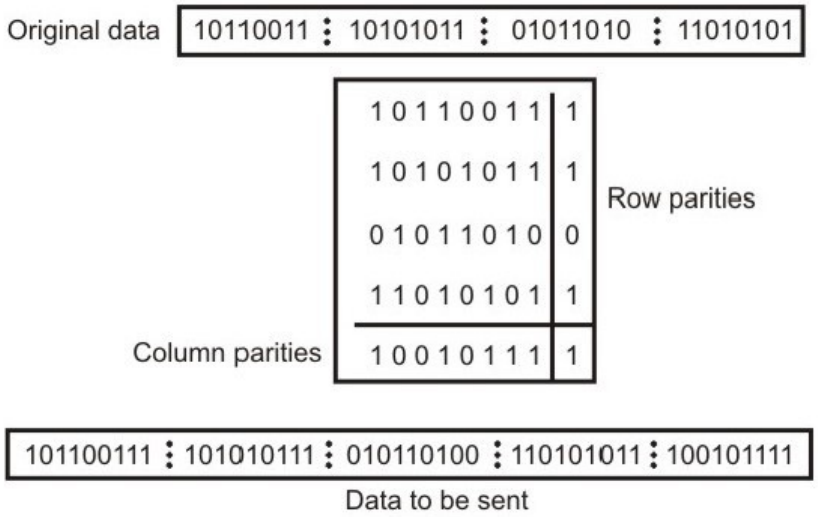
- Parity can only detect single-bit errors
- Parity is not effective against certain types of errors, such as burst errors, where multiple consecutive bits are corrupted



2D Parity / LRC

Two-dimensional (2D) parity is an extension of the basic parity checking concept, where parity bits are added both horizontally and vertically to a block of data.

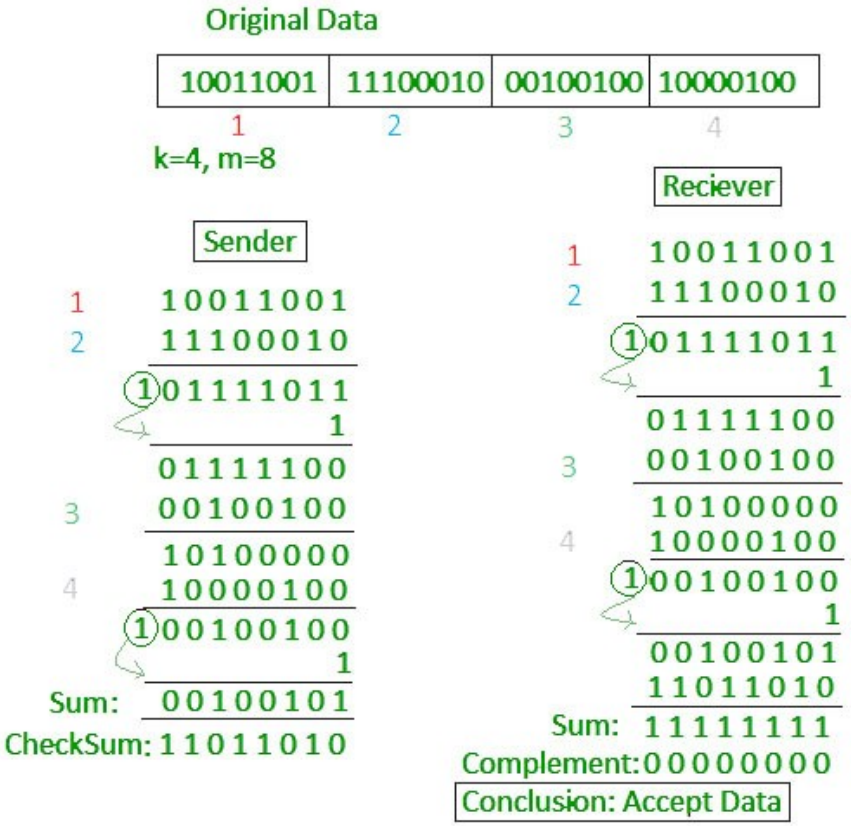
- 2D parity is still primarily focused on detecting single-bit errors.
- 2D parity is not effective in detecting or correcting burst errors, where multiple bits in close proximity are corrupted.



Checksum

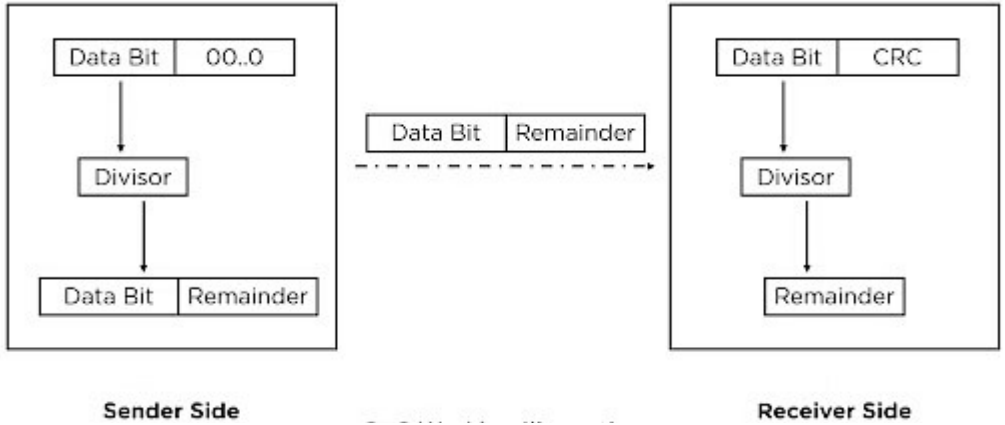
Checksums are a form of error-checking technique commonly used in computing and data communication to detect errors in data transmission.

- Checksums are not effective in detecting more sophisticated errors, such as burst errors or multiple simultaneous errors.
- Many checksum algorithms require fixed-size blocks of data. If the data size varies, padding or other adjustments may be necessary, potentially leading to inefficiencies in terms of both space and computation

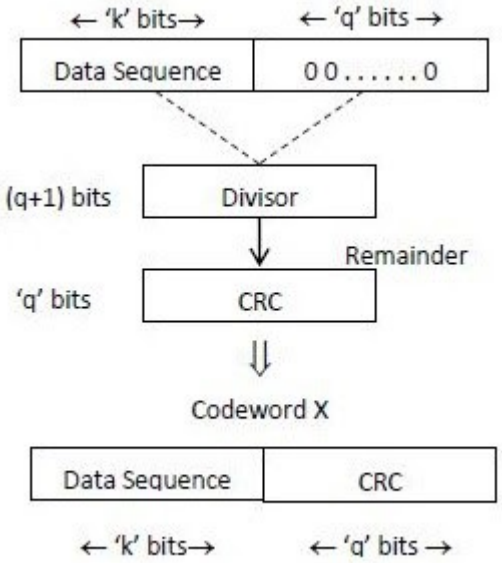


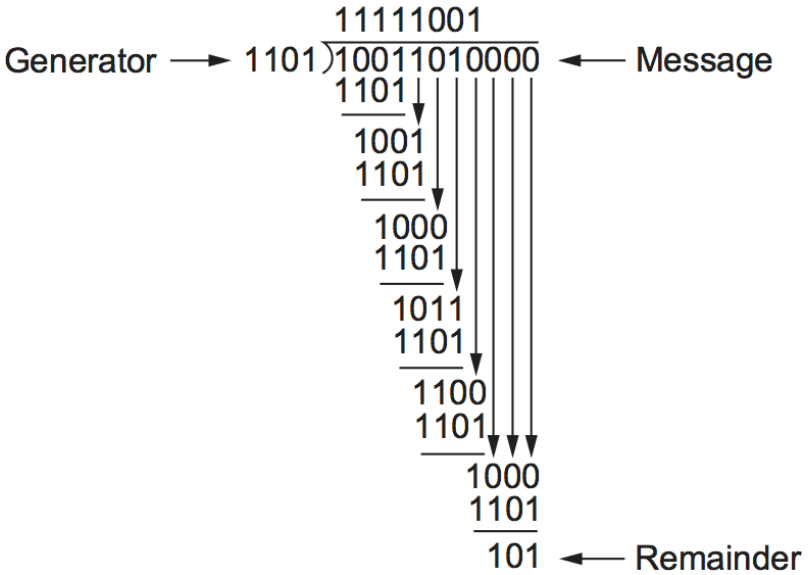
Cyclic Redundancy Check

It is a type of checksum that involves the use of polynomial codes to generate a fixed-size value (CRC code) based on the data being transmitted. This CRC code is then sent along with the data, and the receiving end uses the same polynomial code to recalculate the CRC code. If the recalculated CRC code matches the one received, the data is assumed to be free of errors; otherwise, an error is detected.



CRC Calculation using Polynomial long division





Divisor Polynomial

CRC-32: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

CRC-16: $x^{16} + x^{15} + x^2 + 1$

The selection of a divisor polynomial in Cyclic Redundancy Check (CRC) is a crucial aspect that directly influences the effectiveness of error detection.

- **Degree of Polynomial** : The degree of the divisor polynomial should be carefully chosen based on the desired level of error detection. A higher-degree polynomial can detect a broader range of errors, including longer burst errors. However, it also results in a longer CRC code.
- **Odd Degree Polynomial**: To ensure that the divisor polynomial can detect all single-bit errors, it is common to use a polynomial with an odd degree. An odd-degree polynomial ensures that it can detect any odd number of errors, including a single-bit error.
- **Performance Considerations**: The computational complexity of polynomial division increases with the degree of the polynomial. Therefore, the selection of a divisor polynomial should also consider the computational efficiency, especially in systems where real-time processing is crucial.

Hamming Code - Error Correction

In this technique, additional parity bits are added to the message, allowing the receiver to detect and correct single-bit and multiple-bit errors.

Example: **Suppose the sender wants to transmit the message whose bit representation is ‘1011001’**

Total number of bits (d) = 7

Total number of redundant parity bits (p) = 4

Thus, total bits (d+p) = 7 + 4 = 11

For computing the total number of redundant bits: $2^p \geq d+p+1$

DATA + REDUNDANT BITS

CODE WORD

1: Parity Bits Addition

Parity bits are inserted at positions that are powers of 2

-	-	-	2^3	-	-	-	2^2	-	2^1	2^0	Power of 2
1	0	1	P8	1	0	0	P4	1	P2	P1	Message (bit sequence)
11	10	9	8	7	6	5	4	3	2	1	Total bits (d+p)

2: Calculating Parity Bits

Each parity bit checks a specific set of bits in the data word, including other parity bits. The parity bits are set such that the number of 1s in the positions they check (including the parity bit itself) is even or odd, depending on the chosen parity scheme (usually even parity).

For example, parity bit 1 checks bits 1, 3, 5, 7, etc., parity bit 2 checks bits 2, 3, 6, 7, etc., and so on.

Position	P8	P4	P2	P1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1

3: Error Detection

11	10	9	8	7	6	5	4	3	2	1	Bit Position
D7	D6	D5	P8	D4	D3	D2	P4	D1	P2	P1	Hamming Code Pattern
1	0	1	0	1	0	0	1	1	1	0	Original Data to be sent

P1 : 3, 5, 7, 9, 11

P2 : 3, 6, 7, 10, 11

P4 : 5, 6, 7

P8 : 9, 10, 11

3: Error Detection

11	10	9	8	7	6	5	4	3	2	1	Bit Position
D7	D6	D5	P8	D4	D3	D2	P4	D1	P2	P1	Hamming Code Pattern
1	0	1	0	1	0	0	1	1	1	0	Original Data to be sent
1	0	0	0	1	0	0	1	1	1	0	Transmitted data with error

P1 : 3, 5, 7, 9, 11

P2 : 3, 6, 7, 10, 11

P4 : 5, 6, 7

P8 : 9, 10, 11

4: Error Correction

Bit Position	11	10	9	P8 8	7	6	5	P4 4	3	P2 2	P1 1
Data Received	1	0	0	0	1	0	0	1	1	1	0

3,5,7,9,11	P1	1
3,6,7,10,11	P2	1
5,6,7	P4	1
9,10,11	P8	1

Hamming Weight

The **Hamming weight** of a codeword is equal to the number of non-zero elements in the codeword.

Hamming weight of a codeword c is denoted by **w(c).**

w(0100)=1 and **w(1111)**=4

Hamming Distance

Hamming distance between two codewords is the number of places by which the codewords differ. For two codes $\mathbf{c_1}$ and $\mathbf{c_2}$, the hamming distance is denoted as $\boldsymbol{d(c_1, c_2)}$

E.g.: c1 ⊕ c2 ⇒ 0100 ⊕ 1111 ⇒ 1011. The XOR operator gives result 3 as there are three 1s obtained after XORing of 0s and 1s.

Min. Hamming Distance

For two codewords of same length, it is define as the *minimum or smallest hamming distance between the two codewords*. It is calculated by counting the number of positions in which the corresponding symbols are different. It is denoted by ***d**** or d_{min}.

E.g.: consider four codewords as: c_1**=010,** c_2**=011,** c_3**=111,** c_4**=101**. The hamming distance between codewords is:

Min. Hamming Distance **c_1=010, c_2=011, c_3=111, c_4=101.**

$010 \oplus 011 = 001, d(010, 011) = 1$

$010 \oplus 111 = 101, d(010, 111) = 2$

$010 \oplus 101 = 111, d(010, 101) = 3$

$011 \oplus 111 = 100, d(011, 111) = 1$

$011 \oplus 101 = 110, d(011, 101) = 2$

$111 \oplus 101 = 010, d(111, 101) = 1$

Smallest non-zero weight = 3

Therefore:
min. hamming distance $d_{min} = 3$

SO, FOR WHOLE CODEWORD

Maximum detectable errors = $d_{min} - 1$ 2 bit

Maximum correctable errors = $(d_{min} - 1) / 2$ 1 bit

4 BIT	HAMMING CODE	WEIGHT
0000	0000000	0
0001	0000111	3
0010	0011001	3
0011	0011110	4
0100	0101010	3
0101	0101101	4
0110	0110011	4
0111	0110100	3
1000	1001011	4
1001	1001100	3
1010	1010010	3
1011	1010101	4
1100	1100001	3
1101	1100110	4
1110	1111000	3
1111	1111111	7

Example / Test

DATA : 1111

Codeword : 1111111

Error during Tx : 1111100

Calculate answer ?

Parity-check matrix H

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

P1
P2
P4

1 2 3 4 5 6 7
AFFECTED BIT POSITION

Single Error Correction - Double Error Detection (SEC-DED)

```
HAMMING CODE (SEC-DED) LOGIC
============================================================
SYNDROME | GLOBAL PARITY | RESULT           | ACTION
---------|---------------|------------------|---------------
000      | Even (Valid)  | No Errors        | Accept Data
Not 000  | Odd (Invalid) | Single Bit Error | Correct Bit S
Not 000  | Even (Valid)  | Double Bit Error | REJECT DATA
000      | Odd (Invalid) | P-bit Error      | Fix P-bit
============================================================

EXAMPLE (1111100 1)
------------------------------------------------------------
1. Syndrome Check: Result is 011 (Points to Bit 3)
2. Global Check:   Result is Even (Parity looks "Correct")
3. LOGIC:          Non-Zero Syndrome + Even Parity
4. CONCLUSION:     Double Bit Error Detected (Cannot Fix)
------------------------------------------------------------
```

Next Lecture: OSI Model